Contents

Contents 1

Appendix A Computations in GAP 1
A.1 16-cell . . . 1
A.1.1 Cycles of the 16-cell . . . 1
A.1.2 Candidates to be S_2 of the the 16-cell . . . 3
A.1.3 A regular polyhedron with helical faces of the 16-cell . . . 3
A.2 8-cell and 24-cell . . . 4
A.2.1 Cycles of the 24-cell . . . 4
A.3 120-cell . . . 6
A.3.1 Cycles of the 120-celll . . . 6
A.3.2 Candidates to be S_2 of the 120-cell for S_1 skews . . . 8
A.3.3 Candidates to be S_2 of the 120-cell for S_1 helical . . . 9
A.4 600-cell . . . 11
A.4.1 Cycles of the 600-cell . . . 11
A.4.2 Candidates to be S_2 of the 600-cell for S_1 skews . . . 15
A.4.3 Candidates to be S_2 of the 600-cell for S_1 helical . . . 17
A.4.4 A chiral polyhedron from the 600-cell with helical faces . . . 20
A.5 4-polytopes . . . 21
A.5.1 Searching S_3 in the 5-cell . . . 26
A.5.2 Searching S_3 in the 8-cell . . . 27
A.5.3 Searching S_3 in the 24-cell . . . 30
A.5.4 Searching S_3 in the 600-cell . . . 33
A.5.5 Searching S_3 in the 120-cell . . . 35

Bibliography 39

APPENDIX A

Computations in GAP

The calculations were performed in GAP ([4]).

A.1 16-cell

A.1.1 Cycles of the 16-cell

Distinguished generators of the 16-cell:

```
gap> r0:=(1,2)(5,6);
(1,2)(5,6)
gap> r1:=(2,3)(6,7);
(2,3)(6,7)
gap> r2:=(3,4)(7,8);
(3,4)(7,8)
gap> r3:=(4,8);
(4,8)
gap> P:=Group(r0,r1,r2,r3);
Group([ (1,2)(5,6), (2,3)(6,7), (3,4)(7,8), (4,8) ])
gap> eGC:=Elements(P);;
```

We will remove the involutions and the symmetries of order three (because these correspond to the triangular faces of the 16-cell and these cycles are in a plane).

```
gap> filP:=[];; ##Filter of P.
gap> for j in [1..Length(eGC)] do
>       if Order(eGC[j])=2 or Order(eGC[j])=3 then
>               continue;
>       else
>               Add(filP,eGC[j]);
>       fi;
> od;
gap> filP;;
gap> Length(filP);
277
gap> #Since the base edge is {1, 2} and the base vertex is 1, then we want a rotation
# that sends 1 to 2 when it rotates the first time.
```

```
gap> pHel:=[];;
gap> for i in [1..Length(filP)] do
>       if Length(Cycle(filP[i],1))>1 and Cycle(filP[i],1)[2]=2 then
>               Add(pHel,filP[i]);
>       fi;
> od;
gap> pHel;
[ (1,2)(3,4,7,8)(5,6), (1,2)(3,8,7,4)(5,6), (1,2,3)(4,8)(5,6,7),
  (1,2,3,4)(5,6,7,8), (1,2,3,4,5,6,7,8), (1,2,3,5,6,7),
  (1,2,3,5,6,7)(4,8), (1,2,3,8,5,6,7,4), (1,2,3,8)(4,5,6,7),
  (1,2,4,3)(5,6,8,7), (1,2,4,7,5,6,8,3), (1,2,4,5,6,8),
  (1,2,4,3,5,6,8,7), (1,2,4,7)(3,5,6,8), (1,2,4)(3,7)(5,6,8),
  (1,2,4,5,6,8)(3,7), (1,2,5,6), (1,2,5,6)(4,8), (1,2,5,6)(3,4)(7,8),
  (1,2,5,6)(3,4,7,8), (1,2,5,6)(3,7), (1,2,5,6)(3,7)(4,8),
  (1,2,5,6)(3,8,7,4), (1,2,5,6)(3,8)(4,7), (1,2,7,5,6,3),
  (1,2,7,5,6,3)(4,8), (1,2,7,8,5,6,3,4), (1,2,7,8)(3,4,5,6),
  (1,2,7)(3,5,6)(4,8), (1,2,7,4)(3,8,5,6), (1,2,7,4,5,6,3,8),
  (1,2,8,7,5,6,4,3), (1,2,8,3)(4,7,5,6), (1,2,8,5,6,4),
  (1,2,8,7)(3,5,6,4), (1,2,8,3,5,6,4,7), (1,2,8,5,6,4)(3,7),
  (1,2,8)(3,7)(4,5,6) ]
gap> Length(pHel);
38
gap> #The orders of each symmetry will be in PHel.
gap>  PHel:=pHel;;
gap> OrdH:=[];;
gap> for j in [1..Length(pHel)] do
>       OrdH[j]:=Order(pHel[j]);
> od;
gap> OrdH;
[ 4, 4, 6, 4, 8, 6, 6, 8, 4, 4, 8, 6, 8, 4, 6, 6, 4, 4, 4, 4, 4, 4,
  4, 4, 6, 6, 8, 4, 6, 4, 8, 8, 4, 6, 4, 8, 6, 6 ]
```

From the above symmetries, we'll classify those of order 4. And we'll see which ones are conjugate to obtain a representative. Similarly we will do the process for symmetries of order 6 and 8.

```
#First we will classify the previous symmetries of order 4.
gap> SimOrd4:=[];; #Aquí se guardarán los ciclos (S1) de orden 4.
gap> for j in [1..Length(pHel)] do
>       if Order(pHel[j])=4 then
>               Add(SimOrd4,pHel[j]);
>       fi;
> od;
gap> SimOrd4;
[ (1,2)(3,4,7,8)(5,6), (1,2)(3,8,7,4)(5,6), (1,2,3,4)(5,6,7,8),
  (1,2,3,8)(4,5,6,7), (1,2,4,3)(5,6,8,7), (1,2,4,7)(3,5,6,8),
  (1,2,5,6), (1,2,5,6)(4,8), (1,2,5,6)(3,4)(7,8), (1,2,5,6)(3,4,7,8),
  (1,2,5,6)(3,7), (1,2,5,6)(3,7)(4,8), (1,2,5,6)(3,8,7,4),
  (1,2,5,6)(3,8)(4,7), (1,2,7,8)(3,4,5,6), (1,2,7,4)(3,8,5,6),
  (1,2,8,3)(4,7,5,6), (1,2,8,7)(3,5,6,4) ]
gap> Length(SimOrd4);
18
gap> ElConj4:=[];;
gap>    for j in [2..Length(SimOrd4)] do
>               for k in [1..Length(eGC)] do
>                       if  eGC[k]*SimOrd4[1]*eGC[k]^-1= SimOrd4[j] then
>                               Add(ElConj4,
```

```
[j,Factorization(P,SimOrd4[j]),Factorization(P,eGC[k])]);
>                          else
>                                  continue;
>                          fi;
>                  od;
>          od;
gap> ElConj4;; #These are the conjugate symmetries of order 4.
```

A.1.2 Candidates to be S_2 of the the 16-cell

ElFigV returns the symmetry group of the vertex figure of 16-cell. These symmetries fix the base vertex. $v = \{1\}$.

The goal is find polyhedra with skew faces, so we search within the group of symmetries of the vertex figure of 16-cell, the candidate symmetries to construct a skew vertex figure are shown in red, because these are all the symmetries that are not involutions and send the 4 to 2 and those that do not construct skew cycles are discarded ((2,3,4)(6,7,8), (2,7,4)(3,8,6), (2,8,6,4), (2,8,6,4)(3,7)).

On the other hand, in the search for polyhedra with helical faces, all the symmetries that are not involutions and that send 8 to 2 are marked in blue. As we are looking to construct planar vertex figures, the permutations (2,3,4,6,7,8) and (2,7,4,6,3,8) are discarded since they form skew cycles.

```
gap> GFigVer:=Group(r1,r2,r3);
Group([ (2,3)(6,7), (3,4)(7,8), (4,8) ])
gap> ElFigV:=Elements(GFigVer);
```

[(), (4,8), (3,4)(7,8), (3,4,7,8), (3,7), (3,7)(4,8), (3,8,7,4), (3,8)(4,7), (2,3)(6,7), (2,3)(4,8)(6,7), (2,3,4)(6,7,8), (2,3,4,6,7,8), (2,3,6,7), (2,3,6,7)(4,8), (2,3,8,6,7,4), (2,3,8)(4,6,7), (2,4,3)(6,8,7), (2,4,7,6,8,3), (2,4)(6,8), (2,4,6,8), (2,4,3,6,8,7), (2,4,7)(3,6,8), (2,4)(3,7)(6,8), (2,4,6,8)(3,7), (2,6), (2,6)(4,8), (2,6)(3,4)(7,8), (2,6)(3,4,7,8), (2,6)(3,7), (2,6)(3,7)(4,8), (2,6)(3,8,7,4), (2,6)(3,8)(4,7), (2,7,6,3), (2,7,6,3)(4,8), (2,7,8,6,3,4), (2,7,8)(3,4,6), (2,7)(3,6), (2,7)(3,6)(4,8), (2,7,4)(3,8,6), (2,7,4,6,3,8), (2,8,7,6,4,3), (2,8,3)(4,7,6), (2,8,6,4), (2,8)(4,6), (2,8,7)(3,6,4), (2,8,3,6,4,7), (2,8,6,4)(3,7), (2,8)(3,7)(4,6)]

```
gap> Length(ElFigV);
48
```

A.1.3 A regular polyhedron with helical faces of the 16-cell

```
gap> s1:=r0*r1*r2*r3;
(1,8,7,6,5,4,3,2)
gap> s2:=r3*r2*r1*r2;
(2,4,6,8)
```

Calculating the right transversals of $\langle S_1 \rangle$, $\langle S_1 S_2 \rangle$ and $\langle S_2 \rangle$ in $G = \langle S_1, S_2 \rangle$.

```
gap> G:=Group(s1,s2);
Group([ (1,8,7,6,5,4,3,2), (2,4,6,8) ])
gap> Size(G);
32
gap> genS1:=Group(s1);
Group([ (1,8,7,6,5,4,3,2) ])
gap> Elements(RightTransversal(G,genS1));
[ (), (2,4,6,8), (2,6)(4,8), (2,8,6,4) ]
gap> genS1S2:=Group(s1*s2);
Group([ (1,2)(3,4)(5,6)(7,8) ])
gap> Elements(RightTransversal(G,genS1S2));
[ (), (2,4,6,8), (2,6)(4,8), (2,8,6,4), (1,2,3,4,5,6,7,8),
  (1,2,5,6)(3,4,7,8), (1,2,7,8,5,6,3,4), (1,3,5,7)(2,4,6,8),
```

```
  (1,3,5,7)(2,6)(4,8), (1,3,5,7)(2,8,6,4), (1,4,7,2,5,8,3,6),
  (1,4)(2,7)(3,6)(5,8), (1,5)(2,6)(3,7)(4,8), (1,5)(2,8,6,4)(3,7),
  (1,6,3,8,5,2,7,4), (1,7,5,3)(2,8,6,4) ]
gap> genS2:=Group(s2);
Group([ (2,4,6,8) ])
gap> Elements(RightTransversal(G,genS2));
[ (), (1,2,3,4,5,6,7,8), (1,3,5,7)(2,4,6,8), (1,4,7,2,5,8,3,6),
  (1,5)(2,6)(3,7)(4,8), (1,6,3,8,5,2,7,4), (1,7,5,3)(2,8,6,4),
  (1,8,7,6,5,4,3,2) ]
```

A.2 8-cell and 24-cell

A.2.1 Cycles of the 24-cell

Distinguished generators of the 24-cell:

```
gap> r0:=(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)(19,23);
(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)(19,23)
gap> r1:=(2,24)(4,21)(5,19)(7,18)(8,14)(10,17)(12,20)(13,22)(15,23);
(2,24)(4,21)(5,19)(7,18)(8,14)(10,17)(12,20)(13,22)(15,23)
gap> r2:=(5,13)(6,14)(7,15)(8,16)(17,20)(21,24);
(5,13)(6,14)(7,15)(8,16)(17,20)(21,24)
gap> r3:=(2,13)(3,16)(6,9)(7,12)(18,20)(22,24);
(2,13)(3,16)(6,9)(7,12)(18,20)(22,24)
gap> P:=Group(r0,r1,r2,r3);
<permutation group with 4 generators>
gap> eGC:=Elements(P);;
gap> v:=11;
11
gap> e:=10;
10
gap> #We will remove symmetries of order 2 and 3.
gap> filP:=[];; ##Filtro de P.
gap> for j in [1..Length(eGC)] do
>       if (Order(eGC[j])=2 or Order(eGC[j])=3)=true then
>               continue;
>       else
>               Add(filP,eGC[j]);
>       fi;
> od;
gap> filP;;
gap> Length(filP);
933
gap> #The base vertex is 11, because it is a fixed vertex for the three reflections: r1, r2
gap> #The base vertex is 11 and the base edge is {11, 10}, then we want a rotation that sen
when it rotates for the first time.
gap> pHel:=[];; #Possible S1 cycles will be saved here.
gap> for i in [1..Length(filP)] do
>       if Length(Cycle(filP[i],v))>1 and Cycle(filP[i],v)[2]=e
                and Length(Cycle(filP[i],v))=Order(filP[i]) then
>               Add(pHel,filP[i]);
>       fi;
> od;
gap> pHel;
```

```
[ (1,4,3,2)(5,8,7,6)(9,12,11,10)(13,16,15,14)(18,19,22,23),
  (1,4,3,2)(5,16,7,14)(6,13,8,15)(9,12,11,10)(17,20)(18,19,22,23)(21,24),
  (1,4,8,7,6,2)(3,5)(9,15)(10,14,13,16,12,11)(17,18,19,21,22,23),
  (1,4,8,12,11,10,14,2)(3,5,16,7,9,15,6,13)(17,20,18,19,21,24,22,23),
  (1,4,16,12,11,10,6,2)(3,13,8,15,9,7,14,5)(17,18,19,24,21,22,23,20),
  (1,4,16,15,14,2)(3,13)(5,8,12,11,10,6)(7,9)(18,19,24,22,23,20),
  (1,4,8,5)(2,3,7,6)(9,13,16,12)(10,14,15,11)(17,19,21,23),
  (1,4,8,12,9,13)(2,3,7,11,10,14)(5,16)(6,15)(17,20,19,21,24,23),
  (1,4,16,12,9,5)(2,3,15,11,10,6)(7,14)(8,13)(17,19,24,21,23,20),
  (1,4,16,13)(2,3,15,14)(5,8,12,9)(6,7,11,10)(19,24,23,20),
  (1,4,3,7,6,5)(2,8)(9,13,16,15,11,10)(12,14)(17,19,22,21,23,18),
  (1,4,3,7,11,10,9,13)(2,8,15,6,12,14,5,16)(17,20,19,22,21,24,23,18),
  (1,4,16,15,11,10,6,5)(2,8,13,3,12,14,7,9)(17,19,24,22,21,23,20,18),
  (1,4,16,13)(2,8,15,9)(3,12,14,5)(6,7,11,10)(17,18)(19,24,23,20)(21,22),
  (1,4,8,5)(2,16,7,9)(3,12,6,13)(10,14,15,11)(17,19,21,23)(18,20)(22,24),
  (1,4,8,7,11,10,14,13)(2,16,5,3,12,6,15,9)(17,18,20,19,21,22,24,23),
  (1,4,3,15,11,10,9,5)(2,16,7,14,12,6,13,8)(17,19,22,24,21,23,18,20),
  (1,4,3,15,14,13)(2,16)(5,8,7,11,10,9)(6,12)(18,20,19,22,24,23),
  (1,4,24,3,2,21)(5,23,6,15,19,16)(7,18,14,13,22,8)(9,12,17,11,10,20),
  (1,4,22,8,12,17,11,10,18,14,2,21)(3,13,24,16,5,23,9,7,20,6,15,19),
  (1,4,22,8,5,24)(2,21,16,12,17,6)(3,13,23,9,7,19)(10,18,14,15,20,11),
  (1,4,23,11,10,19)(2,21,3,15,18,14)(5,22,8,12,17,9)(6,13,24,16,7,20),
  (1,4,23,11,10,19)(2,21,16,12,17,6)(3,15,20,9,5,24)(7,18,14,13,22,8),
  (1,4,24,8,7,17,11,10,20,14,13,21)(2,22,3,5,23,6,12,18,9,15,19,16),
  (1,4,23,11,10,19)(2,22,3,12,18,9)(5,21,8,15,17,14)(6,13,24,16,7,20),
  (1,4,23,11,10,19)(2,22,3,12,18,9)(5,24,8,7,17,6)(13,21,16,15,20,14),
  (1,4,21,16,15,20,11,10,17,6,5,24)(2,22,3,13,23,14,12,18,9,7,19,8),
  (1,4,24,3,7,18,11,10,20,9,13,22)(2,23,6,15,17,14,12,19,16,5,21,8),
  (1,4,24,8,5,22)(2,23,6,12,19,16)(3,7,17,9,13,21)(10,20,14,15,18,11),
  (1,4,21,3,15,18,11,10,17,9,5,22)(2,23,14,7,20,6,12,19,8,13,24,16),
  (1,4,21,16,13,22)(2,23,14,12,19,8)(3,15,20,9,5,24)(6,7,18,11,10,17),
  (1,4,21,3,2,24)(5,22,16,15,18,6)(7,19,8,13,23,14)(9,12,20,11,10,17),
  (1,4,22,16,12,20,11,10,18,6,2,24)(3,5,21,8,13,23,9,15,17,14,7,19),
  (1,4,22,16,13,21)(2,24,8,12,20,14)(3,5,23,9,15,19)(6,7,17,11,10,18),
  (1,4,23,11,10,19)(2,24,3,7,18,6)(5,21,8,15,17,14)(9,13,22,16,12,20),
  (1,4,23,11,10,19)(2,24,8,12,20,14)(3,7,17,9,13,21)(5,22,16,15,18,6) ]
gap> Length(pHel);
36
gap> #The orders of each symmetry will be in PHel.
gap> PHel:=pHel;;
gap> OrdH:=[];;
gap> for j in [1..Length(pHel)] do
>       OrdH[j]:=Order(pHel[j]);
> od;
gap> OrdH;
[ 4, 4, 6, 8, 8, 6, 4, 6, 6, 4, 6, 8, 8, 4, 4, 8, 8, 6, 6, 12, 6, 6,
  6, 12, 6, 6, 12, 12, 6, 12, 6, 6, 12, 6, 6, 6 ]
```

Note: The programming for classifying conjugate symmetries is analogous to that shown for the 16-cell case.

A.3 120-cell

A.3.1 Cycles of the 120-celll

The base vertex and distinguished generators of the 120-cell are based in the article [2] and appear in [1]. These are the following:

```
gap> rho := (1 + Sqrt(5)) / 2;; #Golden ratio.
gap> #Distinguished generators of the 120-cell as reflection matrices.
gap> r0 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, -1, 0],
>          [0, 0, 0, 1] ];;
gap> r1 := (1/2) * [ [2, 0, 0, 0],
>                    [0, rho, -1, rho - 1],
>                    [0, -1, 1 - rho, rho],
>                    [0, rho - 1, rho, 1] ];;
gap> r2 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, 1, 0],
>          [0, 0, 0, -1] ];;
gap> r3 := (1/2) * [ [rho, 1, 0, -1 + rho],
>                    [1, 1 - rho, 0, -rho],
>                    [0, 0, 2, 0],
>                    [-1 + rho, -rho, 0, 1] ];;
gap> v:=[1+rho,1,-2+rho,0];; #Base vertex.
gap> P:=Group(r0,r1,r2,r3);;
gap> eGC:=Elements(P);;
gap> e:=v*r0;;
gap> OrdC:=[];;
gap> for j in [1..Length(eGC)] do
>        if Order(eGC[j])>1 then
>                Add(OrdC,Order(eGC[j]));
>        fi;
> od;
gap> Set(OrdC); #Length of the cycles.
[ 2, 3, 4, 5, 6, 10, 12, 15, 20, 30 ]
gap> # We will save the cycles that contain v and e (base vertex and edge)
# in Ciclos.
gap> Ciclos:=[];;
gap> Simgc:=[];; # In Simgc we will save the candidate symmetries to be S1.
#We will remove symmetries of order 3 since they correspond to triangular
#faces of the 120-cell.
gap> for j in [1..Length(eGC)] do
>        ciclo:=[];
>        for i in [1..Order(eGC[j])] do
>                Add(ciclo,v*eGC[j]^i);
>        od;
>        if Length(Set(ciclo))>3 and v*eGC[j]^-1=e and
 Length(Set(ciclo))=Order(eGC[j]) then
>                Add(Ciclos,ciclo);
>                Add(Simgc,eGC[j]);
>        fi;
> od;
gap> OCiclos:=[];;
```

```
gap> for j in [1..Length(Ciclos)] do
>       Add(OCiclos,Length(Ciclos[j]));
> od;
gap> Set(OCiclos); # Length of cycles containing v and e.
[ 5, 10, 30 ]
```

Classifyng symmetries with respect to their order. We will use symmetries of order 5 as an example and then obtain their conjugation class to obtain a representative:

```
gap> OSim:=5;;
gap> SimOrd:=[];; #Candidate to be S1 of order 5.
gap> for j in [1..Length(Simgc)] do
>       if Order(Simgc[j])=OSim then
>               Add(SimOrd,Simgc[j]);
>       fi;
> od;
gap> SimOrd;;
gap> Length(SimOrd);
3
gap> Conj:=[];;
gap>    for j in [2..Length(SimOrd)] do
>               for k in [1..Length(eGC)] do
>                       if eGC[k]*SimOrd[1]*eGC[k]^-1= SimOrd[j] then
>                               Add(Conj,[j,Factorization(P,SimOrd[j]),
Factorization(P,eGC[k])]);
>                       else
>                               continue;
>                       fi;
>               od;
>       od;
gap> Conj;; #This list contains the elements of the conjugation class of
#SimOrd[1].
gap> # We chose as representative element number 2 from the previous list,
#which is:
gap> Factorization(P,SimOrd[]);
x1*x2
gap> #So the candidate to be S1 of order 5 is the following:
gap> s1:=R0*R1;;
```

We will obtain the space in which are the vertices of the cycle, for which we will write the following function called espacio with parameters a symmetry S and a vertex v.

```
gap> espacio:= function(S,v)
> A:=[];
> for i in [0..Order(S)-1] do
>       Add(A,v*S^i);
> od;
> B:=[];
> for i in [1..Order(S)] do
>       Add(B,A[i]-v);
> od;
> return(RankMat(B));
> end;
function( S, v ) ... end

gap> s1:=R0*R1;; #Order 5.
gap> espacio(s1,v); # Plane (This correspond to a pentagonal face of the 120-cell).
```

```
2
```

Similarly we classify the elements according to their order and choose a representative of each conjugation class.

```
gap> s1:=R0*R1*R2;; #Order 10.
gap> espacio(s1,v); #Skew.
3

gap> s1:=R0*R1*R2*R3;; #Order 30.
gap> espacio(s1,v); #Helix.
4
```

A.3.2 Candidates to be S_2 of the 120-cell for S_1 skews

$\underline{S_1 = R_0R_1R_2.}$

```
gap> GVerFig:=Group(r1,r2,r3); #Group of the vertex figure of 120-cell
<matrix group with 3 generators>
gap> ElVF:=Elements(GVerFig);;
gap> #Defining S1.
gap> S1:=r0*r1*r2;; #Order 10.
gap> #The candidates for S2 are those that are in the group of the 120-cell vertex
 #figure, we will remove the identity and the elements of order 2 and 3, because
 #these symmetries would represent digons or would be in a plane, respectively.
gap> CandSigma2:=[];; #Candidates to be S2
gap> for j in [1..Length(ElVF)] do
>       if (Order(ElVF[j])=1 or Order(ElVF[j])=2 or Order(ElVF[j])=3)=true then
>               continue;
>       else
>               Add(CandSigma2,ElVF[j]);
>       fi;
> od;
gap> CandSigma2;;
gap> Length(CandSigma2);
6
gap> #Of the possible candidates to be S2 we will discard those that do not satisfy the fol
gap> S2:=[];;
gap> for j in [1..Length(CandSigma2)] do
>       if Order(S1*CandSigma2[j])=2 then
>               Add(S2,Factorization(P,CandSigma2[j]));
>       fi;
> od;
gap> S2;
[ x2*x3*x4*x2*x3, x3*x4*x2 ]

gap> u:=v*S1;;

gap> S2:=R1*R2*R3*R1*R2;;
gap> Order(S2);
4
gap> espacio(S2,u); #Skew.
3

gap> S2:=R2*R3*R1;;
```

```
gap> Order(S2);
4
gap> espacio(S2,u); #Skew.
3
```

A.3.3 Candidates to be S_2 of the 120-cell for S_1 helical

$S_1 = R_0R_1R_2R_3$.

```
gap> S1:=r0*r1*r2*r3;; #Order 30.
gap> CandSigma2:=[];; #Candidates to be S2
gap> for j in [1..Length(ElVF)] do
>       if (Order(ElVF[j])=1 or Order(ElVF[j])=2)=true then
>               continue;
>       else
>               Add(CandSigma2,ElVF[j]);
>       fi;
> od;
gap> CandSigma2;;
gap> Length(CandSigma2);
14
gap> #Of the possible candidates to be S2 we will discard those that do not satisfy
# the following condition: Order(S1*S2)=2;
gap> S2:=[];;
gap> for j in [1..Length(CandSigma2)] do
>       if Order(S1*CandSigma2[j])=2 then
>               Add(S2,Factorization(P,CandSigma2[j]));
>       fi;
> od;
gap> S2;
[ x4*x2*x3*x2, x3*x4*x3*x2, x4*x3*x2 ]

gap> u:=v*S1;;

gap> S2:=r3*r1*r2*r1;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=r2*r3*r2*r1;;
gap> Order(S2);
3
gap> espacio(S2,u);  #Plane.
2

gap> S2:=r3*r2*r1;;
gap> Order(S2);
4
gap> espacio(S2,u); #Skew.
3
```

Searching for the polyhedron $\{30,3\}$ in the list [3].

Group of the first type polyhedron $\{30,3\}$ which appears in [3] listed as :
R97.9 : Type $\{3,30\}_{20}$ of order 2880. We will call L the group of this polyhedron.

```
gap> l:= FreeGroup( "l0", "l1","l2" );;
gap> L:=l/[ l.1^2, l.2^-3, (l.2 * l.3)^2, (l.2 * l.1)^2, (l.3 * l.1)^2,
l.3 * l.2 * l.3^-2 * l.2 * l.3^-2 * l.2 * l.3^-1 * l.2 * l.3^-2 * l.2
* l.3^2 * l.2^-1 * l.3^2, l.3 * l.2 * l.3^-3 * l.2^-1
* l.3^5 * l.2^-1 * l.3^-3 * l.2 * l.3^6 ];
<fp group on the generators [ l0, l1, l2 ]>
gap> Size(L);
2880
gap> cc:=ConjugacyClassesSubgroups(L);;
rs1:=[];; #In rs1 we will save the subgroups of L of order 1440.
gap> for i in [1..Length(cc)] do
>       if Size(Representative(cc[i]))=1440 then
>               Add(rs1,Representative(cc[i]));
>       fi;
> od;
gap> Length(rs1); #There are three subgroups of L of order 1440.
3
```

Group of the second polyhedron of type $\{30,3\}$ which appears in [3] listed as:
R97.10 : Type $\{3,30\}_{40}$ of order 2880. We will name F the group of this polyhedron.

```
gap> #R97.10 :  Type {3,30}_40  Order 2880  mV = 2  mF = 1
gap> f:= FreeGroup( "f0", "f1","f2" );;
gap> F:=f/[ f.3^2, f.1^-3, (f.1 * f.2)^2, (f.1 * f.3)^2, (f.2 * f.3)^2,
 (f.2^-2 * f.1 * f.2^-1)^3 ];
<fp group on the generators [ f0, f1, f2 ]>
gap> Size(F);
2880
gap> ccs:=ConjugacyClassesSubgroups(F);;
gap> rs2:=[];; #In rs2 se will save the subgroups of F of order 1440.
gap> for i in [1..Length(ccs)] do
>       if Size(Representative(ccs[i]))=1440 then
>               Add(rs2,Representative(ccs[i]));
>       fi;
> od;
gap> Length(rs2); #There is only one subgroup of F of order 1440.
1
```

G is the symmetry group of the 120-cell.

```
gap> g:=FreeGroup("r0","r1","r2","r3");
<free group on the generators [ r0, r1, r2, r3 ]>
gap> G:=g/[g.1^2,g.2^2,g.3^2,g.4^2,(g.1*g.2)^5,(g.2*g.3)^3,(g.3*g.4)^3,
(g.1*g.3)^2,(g.1*g.4)^2,(g.2*g.4)^2];
<fp group on the generators [ r0, r1, r2, r3 ]>
gap> Order(G);
14400
gap> r0:=G.1;;
gap> r1:=G.2;;
gap> r2:=G.3;;
gap> r3:=G.4;;
```

We then define S_1 and S_2 and the symmetry group of the polyhedron $\mathsf{P}(\mathsf{S}_1,\mathsf{S}_2;\mathsf{v})$ will be grup303.

```
gap> s1:=r0*r1*r2*r3;
r0*r1*r2*r3
gap> s2:=r3*r2*r1*r3;
r3*r2*r1*r3
gap> grup303:=Group(s1,s2);
Group([ r0*r1*r2*r3, r3*r2*r1*r3 ])
```

Now we will see if any of the subgroups of L (R97.9) of order 1440 is isomorphic to the symmetry group of the polyhedron $\mathsf{P}(\mathsf{S}_1, \mathsf{S}_2)$.

```
gap> LoadPackage("sonata");
gap> IsIsomorphicGroup(rs1[1],grup303);
false
gap> IsIsomorphicGroup(rs1[2],grup303);
false
gap> IsIsomorphicGroup(rs1[3],grup303);
false
```

Now, we will now see if the only subgroup of F (R97.10) of order 1440 is isomorphic to the group of symmetries of the polyhedron $\mathsf{P}(\mathsf{S}_1, \mathsf{S}_2)$.

```
gap> IsIsomorphicGroup(rs2[1],grup303);
true
```

Therefore the subgroup of rotations of F (R97.10) is isomorphic to the group of $\mathsf{P}(S_1, S_2)$.

A.4 600-cell

A.4.1 Cycles of the 600-cell

The base vertex and distinguished generators of the 600-cell are based in the article [2] and appear in [1]. These are the following:

```
gap> rho := (1 + Sqrt(5)) / 2;; #Golden ratio.
gap> #Distinguished generators of the 600-cell as reflection matrices.
gap> R0 := (1/2) * [ [rho, 1, 0, -1 + rho],
>                    [1, 1 - rho, 0, -rho],
>                    [0, 0, 2, 0],
>                    [-1 + rho, -rho, 0, 1] ];;
gap>
gap> R1 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, 1, 0],
>          [0, 0, 0, -1] ];;
gap>
gap> R2 := (1/2) * [ [2, 0, 0, 0],
>                    [0, rho, -1, rho - 1],
>                    [0, -1, 1 - rho, rho],
>                    [0, rho - 1, rho, 1] ];;
gap>
gap> R3 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, -1, 0],
>          [0, 0, 0, 1] ];;
gap> v:=[1,0,0,0];; #Base vertex.
gap> P:=Group(R0,R1,R2,R3);;
```

```
gap> eGC:=Elements(P);;
gap> e:=v*R0;; # Base edge.
gap> OrdC:=[];;
gap> for j in [1..Length(eGC)] do
>       if Order(eGC[j])>1 then
>               Add(OrdC,Order(eGC[j]));
>       fi;
> od;
gap> Set(OrdC); # Length of the cycles.
[ 2, 3, 4, 5, 6, 10, 12, 15, 20, 30 ]
gap> # We will save the cycles that contain v and e (base vertex and edge)
# in Ciclos.
gap> Ciclos:=[];;
gap> Simgc:=[];; # In Simgc we will save the candidate symmetries to be S1.
#We will remove symmetries of order 3 since they correspond to triangular
#faces of the 600-cell.
gap> for j in [1..Length(eGC)] do
>       ciclo:=[];
>       for i in [1..Order(eGC[j])] do
>               Add(ciclo,v*eGC[j]^i);
>       od;
>       if Length(Set(ciclo))>3 and v*eGC[j]^-1=e and
                Length(Set(ciclo))=Order(eGC[j]) then
>               Add(Ciclos,ciclo);
>               Add(Simgc,eGC[j]);
>       fi;
> od;
gap> OCiclos:=[];;
gap> for j in [1..Length(Ciclos)] do
>       Add(OCiclos,Length(Ciclos[j]));
> od;
gap> Set(OCiclos); # Length of cycles containing v and e.
[ 4, 5, 6, 10, 12, 15, 20, 30 ]
gap> #Classifyng symmetries with respect to their order:
gap> #We will use symmetries of order 4 as an example and then obtain their
#conjugation class to obtain a representative:
gap> OSim:=4;;
gap> SimOrd:=[];; #Candidate to be S1 of order 4.
gap> for j in [1..Length(Simgc)] do
>       if Order(Simgc[j])=OSim then
>               Add(SimOrd,Simgc[j]);
>       fi;
> od;
gap> SimOrd;;
gap> Length(SimOrd);
10
gap> Conj:=[];;
gap>    for j in [2..Length(SimOrd)] do
>               for k in [1..Length(eGC)] do
>                       if  eGC[k]*SimOrd[1]*eGC[k]^-1= SimOrd[j] then
>                               Add(Conj,[j,Factorization(P,SimOrd[j]),
Factorization(P,eGC[k])]);
>                       else
>                               continue;
```

```
>                           fi;
>                   od;
>           od;
gap>Conj;; #This list contains the elements of the conjugation class of
#SimOrd[1].
gap> # We chose as representative element number 7 from the previous list,
#which is:
gap> Factorization(P,SimOrd[7]);
x1*x2*x3
gap> #So the candidate to be S1 of order 4 is the following:
gap> s1:=R0*R1*R2;; #Orden 4
```

We will obtain the space in which are the vertices of the cycle, for which we will write the following function called espacio with parameters a symmetry S and a vertex v.

```
gap> espacio:= function(S,v)
> A:=[];
> for i in [0..Order(S)-1] do
>           Add(A,v*S^i);
> od;
> B:=[];
> for i in [1..Order(S)] do
>           Add(B,A[i]-v);
> od;
> return(RankMat(B));
> end;
function( S, v ) ... end

gap> s1:=R0*R1*R2;; #Orden 4
gap> espacio(s1,v); # Skew (This correspond to the Petrie on the base tetrahedron).
3
```

Similarly we classify the elements according to their order and choose a representative of each conjugation class.

```
gap> s1:=R0*R1*R2*R3*R2*R1;; #Order 5
gap> espacio(s1,v); #Plane.
2

gap> s1:=R3*R0*R1*R2*R3*R2*R1;; #Order 6
gap> espacio(s1,v); #Skew.
3

gap> #There are four conjugacy classes for symmetries of order 10. Their
 #respective representatives are s1, sa, sb and sc.

gap> s1:=R0*R1*R2*R3*R1;; #Order 10
gap> A:=[];;
gap> for i in [0..Order(s1)-1] do
>           Add(A,v*s1^i);
> od;
gap> RankMat(A);
4
gap> B:=[];;
gap> for i in [1..Order(s1)] do
>           Add(B,A[i]-v);
> od;
```

```
gap> RankMat(B); #Skew.
3
gap> sa:=R3*R0*(R1*R2*R3)^3*R1*R2*R1;;
gap> A1:=[];;
gap> for i in [0..Order(s1)-1] do
>       Add(A1,v*sa^i);
> od;
gap> RankMat(A1);
2
gap> B1:=[];;
gap> for i in [1..Order(s1)] do
>       Add(B1,A1[i]-v);
> od;
gap> RankMat(B1); #Plane.
2
gap> sb:=R0*(R1*R2*R3)^3*R2*R1;;
gap> B1:=[];;
gap> for i in [0..Order(s1)-1] do
>       Add(B1,v*sb^i);
> od;
gap> sc:=R3*R2*R3*R0*(R1*R2*R3)^3*R1*R2*R1;;
gap> C1:=[];;
gap> for i in [0..Order(s1)-1] do
>       Add(C1,v*sc^i);
> od;
gap> Set(A)=Set(A1);
false
gap> Set(A1)=Set(B1);
true
gap> Set(B1)=Set(C1);
true

gap> #The cycle generated by the representative of the first conjugacy class is
  #skew, while the cycles generated by the respective representatives of the
  #following three conjugacy classes contain the same vertices and those are in
  #a plane.

gap> s1:=R0*(R1*R2*R3)^3;; #Order 12
gap> espacio(s1,v); #Helix.
4

gap> s1:=R2*R3*R0*R1*R2*R3*R2*R1;; #Order 15
gap> espacio(s1,v); #Helix.
4

gap> s1:=R2*R3*R0*R1*R2*R3;; #Order 20
gap> espacio(s1,v); #Helix.
4

gap> s1:=R0*R1*R2*R3;; #Order 30
gap> espacio(s1,v); #Helix.
4
```

A.4.2 Candidates to be S_2 of the 600-cell for S_1 skews

$S_1 = R_0R_1R_2$.

```
gap> GVerFig:=Group(R1,R2,R3); #Group of the vertex figure of 600-cell
<matrix group with 3 generators>
gap> ElVF:=Elements(GVerFig);;
gap> #Defining S1 as S1.
gap> S1:=R0*R1*R2;; #Order 4.
gap> #The candidates for S2 are those that are in the group of the 600-cell
  #vertex figure, we will remove the identity and the elements of order 2 and 3,
  #because these symmetries would represent digons or would be in a plane,
  #respectively.
gap> CandSigma2:=[];; #Candidates to be S2
gap> for j in [1..Length(ElVF)] do
>       if (Order(ElVF[j])=1 or Order(ElVF[j])=2 or Order(ElVF[j])=3)=true then
>               continue;
>       else
>               Add(CandSigma2,ElVF[j]);
>       fi;
> od;
gap> CandSigma2;;
gap> Length(CandSigma2);
68
gap> #Of the possible candidates to be S2 we will discard those that do not
 #satisfy that the order of S1*S2 is 2.
gap> S2:=[];;
gap> for j in [1..Length(CandSigma2)] do
>       if Order(S1*CandSigma2[j])=2 then
>               Add(S2,Factorization(P,CandSigma2[j]));
>       fi;
> od;
gap> #The following symmetries are the candidates to be S2 for S1=R0*R1*R2.
gap> S2;
[ (x2*x3*x4)^2*x3, x3*x4*x2, x3*x4*x2*x3*x4, x2*x3*x4*x2*x3 ]

gap> u:=v*S1;;

gap> S2:=(R1*R2*R3)^2*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
3

gap> S2:=R2*R3*R1;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=R2*R3*R1*R2*R3;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
3
```

```
gap> S2:=R1*R2*R3*R1*R2;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3
```

$\underline{S_1 = R_3R_0R_1R_2R_3R_2R_1.}$
We repeat the same code above to obtain the candidates to be S_2 .

```
S1:=R3*R0*R1*R2*R3*R2*R1;; #Orden 6.
#The following symmetries are the candidates to be S2 for S1=R3*R0*R1*R2*R3*R2*R1.
gap> S2;
[ x2*(x3*x4)^2*x2*x3*x4*x3, x2*(x3*x4)^2*x2*x3, x2*(x3*x4)^2*x2, (x2*x3*x4)^3,
  (x2*x3*x4)^2*x2 ]

gap> u:=v*S1;;

gap> S2:=R1*(R2*R3)^2*R1*R2*R3*R2;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=R1*(R2*R3)^2*R1*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
3

gap> S2:=R1*(R2*R3)^2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R1*R2*R3)^3;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=(R1*R2*R3)^2*R1;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
3
```

$\underline{S_1 = R_0R_1R_2R_3R_1.}$
We repeat the same code above to obtain the candidates to be S_2 .

```
gap> S1:=R0*R1*R2*R3*R1;; #Orden 10 skew.
gap> S2;
[ x4*x3*x4*x2*x3*x4*x3, x4*x2*x3*x2, x4*x3*x2, (x4*x2*x3)^2*x4,
```

```
  x4*x3*x4*x2*x3 ]

gap> u:=v*S1;;

gap> S2:=R3*R2*R3*R1*R2*R3*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew
3

gap> S2:=R3*R1*R2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane
2

gap> S2:=R3*R2*R1;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew
3

gap> S2:=(R3*R1*R2)^2*R3;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew
3

gap> S2:=R3*R2*R3*R1*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew
3
```

A.4.3 Candidates to be S_2 of the 600-cell for S_1 helical

$S_1 = R_0(R_1R_2R_3)^3$.

```
gap> S1:=R0*(R1*R2*R3)^3;; #Orden 12 #Helical
gap> CandSigma2:=[];; #Candidates to be S2
gap> for j in [1..Length(ElVF)] do
>       if (Order(ElVF[j])=1 or Order(ElVF[j])=2)=true then
>               continue;
>       else
>               Add(CandSigma2,ElVF[j]);
>       fi;
> od;
gap> CandSigma2;;
gap> Length(CandSigma2);
88
gap> #Of the possible candidates to be S2 we will discard those that do not satisfy the fol
gap> S2:=[];;
gap> for j in [1..Length(CandSigma2)] do
```

```
>         if Order(S1*CandSigma2[j])=2 then
>                 Add(S2,Factorization(P,CandSigma2[j]));
>         fi;
> od;
gap> S2;
[ x4*x3*x4*x2*x3*x4*x3*x2, x4*(x3*x4*x2)^3, (x4*x2*x3)^3*x2, x4*(x3*x4*x2)^2*x3*x2,
  (x4*x2*x3)^4 ]

gap> u:=v*S1;;

gap> S2:=R3*R2*R3*R1*R2*R3*R2*R1;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*(R2*R3*R1)^3;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R3*R1*R2)^3*R1;;
gap> Order(S2);
3
gap> espacio(S2,u);  #Plane.
2

gap> S2:=R3*(R2*R3*R1)^2*R2*R1;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=(R3*R1*R2)^4;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2
```

$\underline{S_1 = R_2R_3R_0R_1R_2R_3R_2R_1.}$

```
gap> S1:=R2*R3*R0*R1*R2*R3*R2*R1;;#Orden 15
gap> #We repeat the same code above to obtain the candidates to be $S_2$ .
gap> S2;
[ (x2*x3*x4)^3*x3, x2*(x3*x4)^2*x2*x3, x2*(x3*x4)^2*x2*x3*x4, x2*(x3*x4)^2*x2,
  (x2*x3*x4)^2*x2*x3 ]

gap> u:=v*S1;;

gap> S2:=(R1*R2*R3)^3*R2;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane
```

```
2

gap> S2:=R1*(R2*R3)^2*R1*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #3 Skew.
3

gap> S2:=R1*(R2*R3)^2*R1*R2*R3;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=R1*(R2*R3)^2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R1*R2*R3)^2*R1*R2;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2
```

$\underline{S_1 = R_2R_3R_0R_1R_2R_3.}$

```
gap> S1:=R2*R3*R0*R1*R2*R3;; #Order 20.
gap> S2;
[ (x4*x2*x3)^2*x4*x3, x4*x2*x3*x2, x4*x3*x4*x2, (x4*x2*x3)^2, x4*x3*x4*x2*x3 ]

gap> u:=v*S1;;

gap> S2:=(R3*R1*R2)^2*R3*R2;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*R1*R2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*R2*R3*R1;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R3*R1*R2)^2;;
gap> Order(S2);
```

```
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*R2*R3*R1*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
3
```

$\underline{S_1 = R_0R_1R_2R_3.}$

```
gap> S1:=R0*R1*R2*R3;; #Orden 30
gap> S2;
[ (x4*x2*x3)^2*x4*x3, x4*x2*x3*x2, x4*x3*x2, x4*x3*x4*x2, (x4*x2*x3)^2 ]

gap> u:=v*S1;;

gap> S2:=(R3*R1*R2)^2*R3*R2;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*R1*R2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*R2*R1;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=R3*R2*R3*R1;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R3*R1*R2)^2;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2
```

A.4.4 A chiral polyhedron from the 600-cell with helical faces

To prove that the polytope $\mathsf{P}(P_0P_1P_3P_2,\ P_2P_1; v)$ of type $\{12,3\}$that appears in [2] is isomorphic to $\mathsf{P}(S_1, R; v)$, we will look for an element α in $[3,3,5]$, such that $\alpha(P_0P_1P_3P_2)\alpha^{-1} = S_1$ and $\alpha(P_2P_1)\alpha^{-1} = R$.

```
gap> S1:=R0*(R1*R2*R3)^3;;   #Orden 12
gap> P0:=R0;;
```

```
gap> P1:=R1*R2*R3*R2*R1;;
gap> P2:=R3*R2*R3;;
gap> P3:=R2;;
gap> R:=R3*R2*R3*R1*R2*R3*R2*R1;; #Este es el candidato a ser S2.
gap> Alfa:=[];;
gap> for k in [1..Length(eGC)] do
>       if  eGC[k]*P0*P1*P3*P2*eGC[k]^-1=S1 and
eGC[k]*P2*P1*eGC[k]^-1=R then
>               Add(Alfa,Factorization(P,eGC[k]));
>       else
>               continue;
>       fi;
> od;
gap> Alfa;
[ x4*x3*x4*(x2*x3*x4*x1)^13*x2*x3*x1*x2*x1, <identity ...> ]
```

Verifying that the following is satisfied $\alpha v \alpha^{-1} = v$.

```
gap> alfa:=R3*R2*R3*(R1*R2*R3*R0)^13*R1*R2*R0*R1*R0;;
gap> alfa*v*alfa^-1=v;
true
```

To check that $\mathsf{P}(S_1, Q; v)$ and $\mathsf{P}(S_1, R; v)$ are isomorphic polyhedra, we are looking for a $\alpha \in [3, 3, 5]$ such that $\alpha S_1 \alpha^{-1} = S_1^{-1}$ and $\alpha Q \alpha^{-1} = R^{-1}$.

```
gap> Q:=(R3*R1*R2)^3*R1;;
gap> Alfa:=[];;
gap> for k in [1..Length(eGC)] do
>       if  eGC[k]*S1*eGC[k]^-1=S1^-1 and eGC[k]*Q*eGC[k]^-1=R^-1 then
>               Add(Alfa,Factorization(P,eGC[k]));
>       else
>               continue;
>       fi;
> od;
gap> Alfa;
[ x1*x2*(x3*x4)^2*(x2*x3*x4*x1*(x2*x3*x4)^2*x1)^3*x2*x3*x4*x1*x2*x3*x1
*x2*x1, x4*(x3*x4*x2)^3*x3*x4 ]
```

A.5 4-polytopes

First, we will define all the functions that we will use for all cases.

Given the symmetries S_1 and S_2, a group G defined as $\langle R_2, R_3 \rangle$, and P the group of one of the convex regular 4-polytopes, the function `candS3(s1, s2, G, P)` returns the candidates to be S_3.

```
gap> candS3 := function(s1, s2, G, P)
>     local eG, resultado, j;
>
>     eG := Elements(G);
>     resultado := [];
>
>     for j in [1..Length(eG)] do
>         if Order(s1 * s2 * eG[j]) = 2 and Order(s2 * eG[j]) = 2 and
Order(eG[j]) > 2 then
>             Add(resultado, Factorization(P, eG[j]));
>         fi;
>     od;
```

```
>
>     return resultado;
> end;
function( s1, s2, G, P ) ... end
```

The function `StabE(v, e, G)` returns the elements of the group $\langle S_1, S_2, S_3 \rangle$ that stabilize the base edge e, in terms of permutations.

```
gap> StabE:=function(v,e,G)
>       local eP, lStabE, i;
>
>       eP:=Elements(G);
>       lStabE:=[];
>       for i in [1..Length(eP)] do
>               if (Length(Cycle(eP[i],v))>1 and Cycle(eP[i],v)[2]=e and
Length(Cycle(eP[i],e))>1 and Cycle(eP[i],e)[2]=v) then
>                       Add(lStabE,eP[i]);
>               fi;
>               if (Length(Cycle(eP[i],v))=1 and Length(Cycle(eP[i],e))=1) then
>                       Add(lStabE,eP[i]);
>               fi;
>       od;
>       return lStabE;
> end;
function( v, e, G ) ... end
```

Given the base face f, the function `face(f, G)` returns all the possible lists corresponding to the same base face f.

```
gap> face:=function(f,G)
>       local eG, fb, j, f1, f2;
>
>       eG := Elements(G);
>       fb:=[];
>       for j in [1..Length(eG)] do
>               f1:=OnTuples( f, eG[j] );
>               f2:=Reversed(f1);
>               Add(fb,f1);
>               Add(fb,f2);
>       od;
>       return fb;
> end;
function( f, G ) ... end
```

The function `StabF(f, G, GS1)` returns the elements of the group $\langle S_1, S_2, S_3 \rangle$ that stabilize the base edge f, in terms of permutations.

```
gap> StabF:=function(f,G,GS1)
>       local lStabF, i, eG;
>
>       eG := Elements(G);
>       lStabF:= [];
>       for i in [1..Length(eG)] do
>               if OnTuples( f, eG[i] ) in face(f,GS1) then
>                       Add(lStabF,eG[i]);
>               fi;
>       od;
>       return lStabF;
```

```
> end;
function( f, G, GS1 ) ... end
```

The function `celda(f,G,U)` returns the base cell c.

```
gap> celda:=function(f,G,U)
>       local transGU, j, cel;
>
>       transGU :=Elements(RightTransversal(G,U));
>       cel:=[];
>       for j in [1..Length(transGU)] do
>               Add(cel,OnTuples( f, transGU[j] ));
>       od;
>       return cel;
> end;
function( f, G, U ) ... end
```

Given a list fa, the function $nf(fa, k)$ returns a list shifted by k positions.

```
gap> nf := function(fa, k)
>     local ln, i, n;
>     n := Length(fa);
>     ln := [];
>     for i in [1..n] do
>         Add(ln, fa[(i + k - 1) mod n + 1]);
>     od;
>     return ln;
> end;
function( fa, k ) ... end
```

Given a face fa, the function `sf(fa)` returns all the possible lists corresponding to the same base face fa.

```
gap> sf:= function(fa)
>     local bc1, k, f1;
>
>     bc1 := [];
>     Add(bc1, fa);
>     Add(bc1, Reversed(fa));
>
>     for k in [1..(Length(fa)-1)] do
>         f1 := nf(fa, k);
>         Add(bc1, f1);
>         Add(bc1, Reversed(f1));
>     od;
>
>     return bc1;
> end;
function( fa ) ... end
```

The function `StabC(G, Celda, bc)` returns the elements of the group $\langle S_1, S_2, S_3 \rangle$ that stabilize the base cell c, in terms of permutations.

```
gap> StabC:= function(G, Celda, bc)
>     local np, eG, lStabC, i, k, j, cont;
>
>     np := Length(Celda);
>     eG := Elements(G);
>     lStabC := [];
>
```

```
>      for i in [1..Length(eG)] do
>          cont := 0;
>          for k in [1..np] do
>              for j in [1..Length(bc)] do
>                  if OnTuples(Celda[k], eG[i]) in bc[j] then
>                      cont := cont + 1;
>                      break;
>                  fi;
>              od;
>          od;
>          if cont = np then
>              Add(lStabC, eG[i]);
>          fi;
>      od;
>
>      return lStabC;
> end;
function( G, Celda, bc ) ... end
```

The following functions are analogous to the previous ones; they are used for the cases of the 120-cell and the 600-cell, since the groups are given in terms of matrices.

```
gap> Stabe:=function(v,e,G)
>        local eP, lStabE, i;
>
>        eP:=Elements(G);
>        lStabE:=[];
>        for i in [1..Length(eP)] do
>                if v*eP[i]=e and e*eP[i]=v then
>                        Add(lStabE,eP[i]);
>                fi;
>                if v*eP[i]=v and e*eP[i]=e  then
>                        Add(lStabE,eP[i]);
>                fi;
>        od;
>        return lStabE;
> end;
function( v, e, G ) ... end

gap> Stabf:=function(f,G,GS1)
>        local lStabF, i, j, eG, im;
>
>        eG := Elements(G);
>        lStabF:= [];
>        for i in [1..Length(eG)] do
>                im:=[]; #Applying the symmetry eG[i] to f.
>                for j in [1..Length(f)] do
>                        Add(im,f[j]* eG[i]);
>                od;
>                if im in sf(f) then
>                        Add(lStabF,eG[i]);
>                fi;
>        od;
>        return lStabF;
```

```
> end;
function( f, G, GS1 ) ... end

gap> cel:=function(f,G,U)
>       local transGU, i, j, cell, im;
>
>       transGU :=Elements(RightTransversal(G,U));
>       cell:=[];
>       for i in [1..Length(transGU)] do
>               im:=[]; #Applying the symmetry transGU[i] ) to f.
>               for j in [1..Length(f)] do
>                       Add(im,f[j]* transGU[i]);
>               od;
>               Add(cell, im);
>       od;
>       return cell;
> end;
function( f, G, U ) ... end

gap> im:=function(Celda,S)
>       local m, l, ImC, im;
>
>       ImC:=[];
>       for m in [1..Length(Celda)] do
>               im:=[]; #Image of each face.
>               for l in [1..Length(Celda[1])] do
>                       Add(im,Celda[m][l]*S);
>               od;
>               Add(ImC,im);
>       od;
>       return ImC;
> end;
function( Celda, S ) ... end

gap> Stabc:= function(G, Celda, bc)
>     local np, eG, lStabC, i, j, k, cont;
>
>     np := Length(Celda);
>     eG := Elements(G);
>     lStabC := [];
>
>     for i in [1..Length(eG)] do
>         cont := 0;
>         for k in [1..np] do
>             for j in [1..Length(bc)] do
>                if im(Celda,eG[i])[k] in bc[j] then
>                     cont := cont + 1;
>                     break;
>                 fi;
>             od;
>         od;
>         if cont = np then
>             Add(lStabC, eG[i]);
>         fi;
>     od;
```

```
>
>      return lStabC;
> end;
function( G, Celda, bc ) ... end
```

A.5.1 Searching S_3 in the 5-cell

```
gap> r0:=(1,2);;
gap> r1:=(2,3);;
gap> r2:=(3,4);;
gap> r3:=(4,5);;
gap> P:=Group(r0,r1,r2,r3);
Group([ (1,2), (2,3), (3,4), (4,5) ])
gap> GS3:=Group(r2,r3);
Group([ (3,4), (4,5) ])
gap> S1:=r0*r1*r2;
(1,4,3,2)
gap> S2:=r2*r3*r1;
(2,3,5,4)
gap> S2p:=r1*r2*r3*r1*r2;
(2,5,3,4)
gap> candS3(S1,S2,GS3,P);
[ x4*x3 ]
gap> S3:=r3*r2;
(3,4,5)
```

So the only candidate symmetry to be S_3 is $S_3 := R_3R_2$, where $S_1 = R_0R_1R_2$ and $S_2 := R_2R_3R_1$. Now we will define $G := \langle S_1, S_2, S_3 \rangle$.

```
gap> v:=1;
1
gap> G:=Group(S1,S2,S3);
Group([ (1,4,3,2), (2,3,5,4), (3,4,5) ])
gap> Stabv:=Elements(Stabilizer(G,v));
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (2,3), (2,3)(4,5), (2,3,4),
  (2,3,4,5), (2,3,5,4), (2,3,5), (2,4,3), (2,4,5,3), (2,4), (2,4,5),
  (2,4)(3,5), (2,4,3,5), (2,5,4,3), (2,5,3), (2,5,4), (2,5), (2,5,3,4),
  (2,5)(3,4) ]
gap> GS2S3:=Elements(Group(S2,S3));
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (2,3), (2,3)(4,5), (2,3,4),
  (2,3,4,5), (2,3,5,4), (2,3,5), (2,4,3), (2,4,5,3), (2,4), (2,4,5),
  (2,4)(3,5), (2,4,3,5), (2,5,4,3), (2,5,3), (2,5,4), (2,5), (2,5,3,4),
  (2,5)(3,4) ]
gap> e:=2;
2
gap> StabE(v,e,G);
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (1,2), (1,2)(4,5), (1,2)(3,4),
  (1,2)(3,4,5), (1,2)(3,5,4), (1,2)(3,5) ]
gap> Length(StabE(v,e,G));
12
gap> Ge:=Group(S3,S1*S2);
Group([ (3,4,5), (1,2)(4,5) ])
gap> Size(Ge);
6
gap> Elements(Ge);
```

```
[ (), (3,4,5), (3,5,4), (1,2)(4,5), (1,2)(3,4), (1,2)(3,5) ]
gap> StabE(v,e,G)[10];
(1,2)(3,4,5)
gap>  Factorization(G,StabE(v,e,G)[10]);
x2^-1*x3*x2*x1^-1
gap> Factorization(P,StabE(v,e,G)[10]);
x4*x3*x1
```

For $S2p$ definided as $S2p := R_1R_2R_3R_1R_2$.

```
gap> candS3(S1,S2p,GS3,P);
[ x3*x4 ]
gap> S3:=r2*r3;
(3,5,4)
gap> G:=Group(S1,S2p,S3);
Group([ (1,4,3,2), (2,5,3,4), (3,5,4) ])
gap> Stabv:=Elements(Stabilizer(G,v));
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (2,3), (2,3)(4,5), (2,3,4),
  (2,3,4,5), (2,3,5,4), (2,3,5), (2,4,3), (2,4,5,3), (2,4), (2,4,5),
  (2,4)(3,5), (2,4,3,5), (2,5,4,3), (2,5,3), (2,5,4), (2,5), (2,5,3,4),
  (2,5)(3,4) ]
gap> GS2S3:=Elements(Group(S2p,S3));
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (2,3), (2,3)(4,5), (2,3,4),
  (2,3,4,5), (2,3,5,4), (2,3,5), (2,4,3), (2,4,5,3), (2,4), (2,4,5),
  (2,4)(3,5), (2,4,3,5), (2,5,4,3), (2,5,3), (2,5,4), (2,5), (2,5,3,4),
  (2,5)(3,4) ]
gap> StabE(v,e,G);
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (1,2), (1,2)(4,5), (1,2)(3,4),
  (1,2)(3,4,5), (1,2)(3,5,4), (1,2)(3,5) ]
gap> Length(StabE(v,e,G));
12
gap> Ge:=Group(S3,S1*S2p);
Group([ (3,5,4), (1,2)(3,5) ])
gap> Size(Ge);
6
gap> Elements(Ge);
[ (), (3,4,5), (3,5,4), (1,2)(4,5), (1,2)(3,4), (1,2)(3,5) ]
gap> StabE(v,e,G)[10];
(1,2)(3,4,5)
gap> Factorization(G,StabE(v,e,G)[10]);
x2*x3^-1*x2*x1^-1
gap> Factorization(P,StabE(v,e,G)[10]);
x4*x3*x1
```

A.5.2 Searching S_3 in the 8-cell

```
gap> r0:= (1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15);
(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)
gap> r1:= (2,4)(6,8)(10,12)(14,16);
(2,4)(6,8)(10,12)(14,16)
gap> r2:= (2,13)(3,16)(6,9)(7,12);
(2,13)(3,16)(6,9)(7,12)
gap> r3:= (5,13)(6,14)(7,15)(8,16);
(5,13)(6,14)(7,15)(8,16)
```

```
gap> P:=Group(r0,r1,r2,r3);
Group([ (1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15), (2,4)(6,8)(10,12)
  (14,16), (2,13)(3,16)(6,9)(7,12), (5,13)(6,14)(7,15)(8,16) ])
gap> eP:=Elements(P);;
gap> GS3:=Group(r2,r3);
Group([ (2,13)(3,16)(6,9)(7,12), (5,13)(6,14)(7,15)(8,16) ])
gap> eGS3:=Elements(GS3);;
gap> S1:=r0*r1*r2*r3;
(1,5,9,10,11,15,3,4)(2,8,13,6,12,14,7,16)
gap> S2:=r3*r2*r1*r3;
(2,5,4)(3,6,8)(9,16,14)(10,12,15)
gap> S2p:=r3*r2*r1*r2;
(3,14,6)(4,13,5)(7,15,10)(8,16,9)
gap> v:=11;
11
gap> e:=10;
10
gap> candS3(S1,S2,GS3,P);
[ x3*x4 ]
gap> S3:=r2*r3;
(2,5,13)(3,8,16)(6,9,14)(7,12,15)

gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true

gap> StabE(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> StabE(v,e,G)=Ge;
true

gap> f:=Cycle(S1,v);  #Base face.
[ 11, 15, 3, 4, 1, 5, 9, 10 ]
gap> GS1:=Group(S1);; #Group generated by S1.
gap> StabF(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> StabF(f,G,GS1)=Gf;
true

gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=celda(f,GS1S2,GS1);  #Faces of the base cell.
[ [ 11, 15, 3, 4, 1, 5, 9, 10 ], [ 11, 12, 8, 5, 1, 2, 14, 15 ],
  [ 11, 10, 6, 2, 1, 4, 16, 12 ], [ 12, 16, 13, 14, 2, 6, 7, 8 ],
  [ 9, 5, 8, 7, 3, 15, 14, 13 ], [ 9, 13, 16, 4, 3, 7, 6, 10 ] ]
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> StabC(G,Celda,bc)=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ () ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ () ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Group size:

```
gap> Size(G);
192
gap> Size(Group(S2,S3));
12
gap> Size(Group(S3,S1*S2));
6
gap> Size(Group(S1,S2*S3));
16
gap> Size(Group(S1,S2));
48
```

Therefore, the 4-polytope found has 16 vertices, 32 edges, 12 faces, and 4 cells.

```
gap> S2:=S2p;
(3,14,6)(4,13,5)(7,15,10)(8,16,9)
gap> candS3(S1,S2,GS3,P);
[ x4*x3 ]
gap> S3:=r3*r2;
(2,13,5)(3,16,8)(6,14,9)(7,15,12)

gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true

gap> StabE(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> StabE(v,e,G)=Ge;
true

gap> f:=Cycle(S1,v);  #Base face.
[ 11, 15, 3, 4, 1, 5, 9, 10 ]
gap> GS1:=Group(S1);; #Group generated by S1.
gap> StabF(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> StabF(f,G,GS1)=Gf;
true

gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=celda(f,GS1S2,GS1);  #Faces of the base cell.
[ [ 11, 15, 3, 4, 1, 5, 9, 10 ], [ 11, 7, 6, 5, 1, 13, 16, 15 ],
  [ 11, 10, 14, 13, 1, 4, 8, 7 ], [ 12, 8, 4, 3, 2, 14, 10, 9 ],
  [ 12, 9, 5, 6, 2, 3, 15, 16 ], [ 12, 16, 13, 14, 2, 6, 7, 8 ] ]
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
```

```
> od;
gap> eGc:=Elements(GS1S2);;
gap> StabC(G, Celda, bc)=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ () ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ () ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Group size:

```
gap> Size(G);
192
gap> Size(Group(S2,S3));
12
gap> Size(Group(S3,S1*S2));
6
gap> Size(Group(S1,S2*S3));
16
gap> Size(Group(S1,S2));
48
```

Therefore, the 4-polytope found has 16 vertices, 32 edges, 12 faces, and 4 cells.

A.5.3 Searching S_3 in the 24-cell

```
gap> r0:=(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)(19,23);
(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)(19,23)
gap> r1:=(2,24)(4,21)(5,19)(7,18)(8,14)(10,17)(12,20)(13,22)(15,23);
(2,24)(4,21)(5,19)(7,18)(8,14)(10,17)(12,20)(13,22)(15,23)
gap> r2:=(5,13)(6,14)(7,15)(8,16)(17,20)(21,24);
(5,13)(6,14)(7,15)(8,16)(17,20)(21,24)
gap> r3:=(2,13)(3,16)(6,9)(7,12)(18,20)(22,24);
(2,13)(3,16)(6,9)(7,12)(18,20)(22,24)
gap> P:=Group(r0,r1,r2,r3);
<permutation group with 4 generators>
gap> eP:=Elements(P);;
gap> GS3:=Group(r2,r3);
Group([ (5,13)(6,14)(7,15)(8,16)(17,20)(21,24), (2,13)(3,16)(6,9)(7,12)
  (18,20)(22,24) ])
gap> eGS3:=Elements(GS3);;
gap> Length(eGS3);
6
gap> S1:=r0*r1*r3*r2*r1*r2;
(1,2,6,10,11,12,16,4)(3,5,14,7,9,15,8,13)(17,20,23,22,21,24,19,18)
gap> S2:=r2*r1*r2*r3*r1*r2;
(2,4,5)(3,8,6)(9,14,16)(10,15,12)(17,18,23)(19,21,22)
gap> S2p:=r1*r2*r3*r1;
(2,4,13)(3,16,14)(6,8,9)(7,12,10)(18,23,20)(19,24,22)
gap>
gap> v:=11;
11
```

```
gap> e:=10;
10
gap> candS3(S1,S2,GS3,P);
[ x3*x4 ]
gap> S3:=r2*r3;
(2,13,5)(3,16,8)(6,14,9)(7,15,12)(17,18,20)(21,22,24)
gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true

gap> StabE(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> StabE(v,e,G)=Ge;
true

gap> f:=Cycle(S1,v);
[ 11, 12, 16, 4, 1, 2, 6, 10 ]
gap> GS1:=Group(S1);; #Group generated by S1.
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> StabF(f,G,GS1)=Gf;
true

gap> GS1S2:=Group(S1,S2); #Group generated by S1 and S2.
Group([ (1,2,6,10,11,12,16,4)(3,5,14,7,9,15,8,13)(17,20,23,22,21,24,19,18),
  (2,4,5)(3,8,6)(9,14,16)(10,15,12)(17,18,23)(19,21,22) ])
gap> Celda:=celda(f,GS1S2,GS1);  #Faces of the base cell.
[ [ 11, 12, 16, 4, 1, 2, 6, 10 ], [ 11, 10, 9, 5, 1, 4, 3, 15 ],
  [ 11, 15, 14, 2, 1, 5, 8, 12 ], [ 12, 8, 7, 6, 2, 14, 13, 16 ],
  [ 9, 10, 6, 7, 3, 4, 16, 13 ], [ 9, 13, 14, 15, 3, 7, 8, 5 ] ]
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> StabC(G,Celda,bc)=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ () ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ () ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Group size:

```
gap> Size(G);
192
gap> Size(Group(S2,S3));
12
gap> Size(Group(S3,S1*S2));
6
gap> Size(Group(S1,S2*S3));
```

```
16
gap> Size(Group(S1,S2));
48
```

Therefore, the 4-polytope found has 16 vertices, 32 edges, 12 faces, and 4 cells.

```
gap> S2:=S2p;
(2,4,13)(3,16,14)(6,8,9)(7,12,10)(18,23,20)(19,24,22)
gap> candS3(S1,S2,GS3,P);
[ x4*x3 ]
gap> S3:=r3*r2;
(2,5,13)(3,8,16)(6,9,14)(7,12,15)(17,20,18)(21,24,22)

gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true

gap> StabE(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> StabE(v,e,G)=Ge;
true

gap> f:=Cycle(S1,v);  #Base face.
[ 11, 12, 16, 4, 1, 2, 6, 10 ]
gap> GS1:=Group(S1);;
gap> StabF(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> StabF(f,G,GS1)=Gf;
true

gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=celda(f,GS1S2,GS1);  #Faces of the base cell.
[ [ 11, 12, 16, 4, 1, 2, 6, 10 ], [ 11, 10, 14, 13, 1, 4, 8, 7 ],
  [ 11, 7, 3, 2, 1, 13, 9, 12 ], [ 12, 9, 5, 6, 2, 3, 15, 16 ],
  [ 9, 13, 14, 15, 3, 7, 8, 5 ], [ 10, 6, 5, 8, 4, 16, 15, 14 ] ]
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> StabC(G,Celda,bc)=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ () ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ () ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Group size:

```
gap> Size(G);
192
gap> Size(Group(S2,S3));
12
gap> Size(Group(S3,S1*S2));
6
gap> Size(Group(S1,S2*S3));
16
gap> Size(Group(S1,S2));
48
```

Therefore, the 4-polytope found has 16 vertices, 32 edges, 12 faces, and 4 cells.

A.5.4 Searching S_3 in the 600-cell

```
gap> rho := (1 + Sqrt(5)) / 2;; #Razn dorada.
gap> #Matrices generadoras del 600-celda.
gap> R0 := (1/2) * [ [rho, 1, 0, -1 + rho],
>                    [1, 1 - rho, 0, -rho],
>                    [0, 0, 2, 0],
>                    [-1 + rho, -rho, 0, 1] ];;
gap>
gap> R1 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, 1, 0],
>          [0, 0, 0, -1] ];;
gap>
gap> R2 := (1/2) * [ [2, 0, 0, 0],
>                    [0, rho, -1, rho - 1],
>                    [0, -1, 1 - rho, rho],
>                    [0, rho - 1, rho, 1] ];;
gap>
gap> R3 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, -1, 0],
>          [0, 0, 0, 1] ];;
gap> v:=[1,0,0,0];; #Vrtice base.
gap> P:=Group(R0,R1,R2,R3);;
gap> eGC:=Elements(P);;
gap> e:=v*R0;;
gap> GS3:=Group(R2,R3);
<matrix group with 2 generators>
gap> eGS3:=Elements(GS3);;
gap> S1:=R0*(R1*R2*R3)^3;;
gap> S2:=R3*R2*R3*R1*R2*R3*R2*R1;;
gap> S2p:=(R3*R1*R2)^3*R1;;

gap> candS3(S1,S2,GS3,P);
[ (x3*x4)^2 ]
gap> S3:=(R2*R3)^2;;

gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
```

```
gap> Stabe(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> Stabe(v,e,G)=Ge;
true
```

f will be the base face.

```
f:=[];;
for i in [0..Order(S1)-1] do
        Add(f,v*S1^i);
od;
GS1:=Group(S1);;
gap> Stabf(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> Stabf(f,G,GS1)=Gf;
true

gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=cel(f,GS1S2,GS1);; #Faces of the base cell.
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> stabC:=Stabc(G,Celda,bc);;
gap> stabC=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Group size.

```
gap> Size(G);
7200
gap> Size(Group(S2,S3));
60
gap> Size(Group(S3,S1*S2));
10
gap> Size(Group(S1,S2*S3));
24
gap> Size(Group(S1,S2));
144
```

Therefore, the 4-polytope found has 120 vertices, 720 edges, 300 faces, and 50 cells.

```
gap> S2:=S2p;;
gap> candS3(S1,S2,GS3,P);
[ (x4*x3)^2 ]
gap> S3:=(R3*R2)^2;;
```

```
gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true

gap> Stabe(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> Stabe(v,e,G)=Ge;
true

gap> f:=[];;
gap> for i in [0..Order(S1)-1] do
>       Add(f,v*S1^i);
> od;
gap> GS1:=Group(S1);; #Group generated by S1.
gap> Stabf(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> Stabf(f,G,GS1)=Gf;
true

gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=cel(f,GS1S2,GS1);; #Faces of the base cell.
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> stabC:=Stabc(G,Celda,bc);;
gap> stabC=eGc;
true
```

Group size.

```
gap> Size(G);
7200
gap> Size(Group(S2,S3));
60
gap> Size(Group(S3,S1*S2));
10
gap> Size(Group(S1,S2*S3));
24
gap> Size(Group(S1,S2));
144
```

Therefore, the 4-polytope found has 120 vertices, 720 edges, 300 faces, and 50 cells.

A.5.5 Searching S_3 in the 120-cell

```
gap> rho := (1 + Sqrt(5)) / 2;; #Razon dorada.
gap> # Matrices generadoras del 120-celda.
gap> r0 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, -1, 0],
>          [0, 0, 0, 1] ];;
gap> r1 := (1/2) * [ [2, 0, 0, 0],
```

```
>                  [0, rho, -1, rho - 1],
>                  [0, -1, 1 - rho, rho],
>                  [0, rho - 1, rho, 1] ];;
gap> r2 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, 1, 0],
>          [0, 0, 0, -1] ];;
gap> r3 := (1/2) * [ [rho, 1, 0, -1 + rho],
>                  [1, 1 - rho, 0, -rho],
>                  [0, 0, 2, 0],
>                  [-1 + rho, -rho, 0, 1] ];;
gap> v:=[1+rho,1,-2+rho,0];; #Vertice base.
gap> P:=Group(r0,r1,r2,r3);;
gap> eGC:=Elements(P);;
gap> e:=v*r0;;
gap> GS3:=Group(r2,r3);
<matrix group with 2 generators>
gap> eGS3:=Elements(GS3);;

gap> S1:=r0*r1*r2*r3;;
gap> S2p:=r3*r2*r1*r2;;
gap> candS3(S1,S2,GS3,P);
[ x3*x4 ]
gap> S3:=r2*r3;;
gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true
gap>
gap> Stabe(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> Stabe(v,e,G)=Ge;
true
gap>
gap> f:=[];;
gap> for i in [0..Order(S1)-1] do
>       Add(f,v*S1^i);
> od;
gap> GS1:=Group(S1);; #Group generated by S1.
gap> Stabf(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> Stabf(f,G,GS1)=Gf;
true
gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=cel(f,GS1S2,GS1);; #Faces of the base cell.
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> stabC:=Stabc(G,Celda,bc);;
gap> stabC=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Group size.

```
gap> Size(G);
7200
gap> Size(Group(S2,S3));
12
gap> Size(Group(S3,S1*S2));
6
gap> Size(Group(S1,S2*S3));
60
gap> Size(Group(S1,S2));
1440
```

Therefore, the 4-polytope found has 600 vertices, 1200 edges, 120 faces, and 5 cells.

```
gap> S2:=S2p;;
gap> S3:=r3*r2;;
gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true
gap>
gap> Stabe(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> Stabe(v,e,G)=Ge;
true
gap>
gap> f:=[];;
gap> for i in [0..Order(S1)-1] do
>       Add(f,v*S1^i);
> od;
gap> GS1:=Group(S1);; #Group generated by S1.
gap> Stabf(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> Stabf(f,G,GS1)=Gf;
true
gap>
gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=cel(f,GS1S2,GS1);; #Faces of the base cell.
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> stabC:=Stabc(G,Celda,bc);;
gap> stabC=eGc;
```

```
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Group size.

```
gap> Size(G);
7200
gap> Size(Group(S2,S3));
12
gap> Size(Group(S3,S1*S2));
6
gap> Size(Group(S1,S2*S3));
60
gap> Size(Group(S1,S2));
1440
```

Therefore, the 4-polytope found has 600 vertices, 1200 edges, 120 faces, and 5 cells.

Bibliography

[1] J. Bracho. Chiral polyhedra in the 3-sphere. https://www.matem.unam.mx/~roli/PetrieCoxeter.html, 2021.

[2] J. Bracho, I. Hubard, and D. Pellicer. Chiral polyhedra in 3-dimensional geometries and from a petrie-coxeter construction. *Discret. Comput. Geom.*, 66:1025–1052, 2021.

[3] M. Conder. Regular orientable maps of genus 2 to 301. https://www.math.auckland.ac.nz/~conder/RegularOrientableMaps301.txt, 2012.

[4] The GAP Group. *GAP – Groups, Algorithms, and Programming, Version 4.8.10*, 2018.